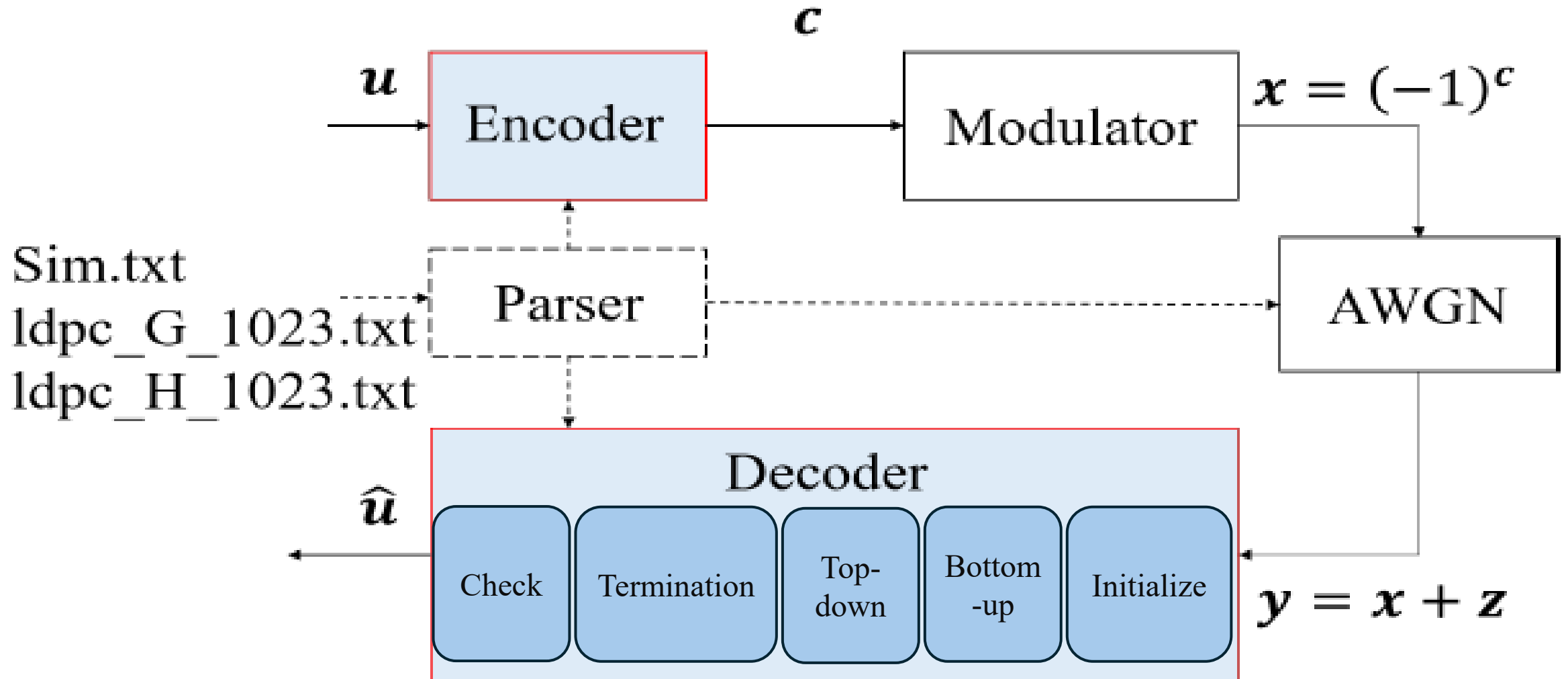


期末Project LDPC Code

F64106016 張博盛

Block diagram of LDPC Code



Read file

```
char ch;
int i = 0, j = 0; // 初始化行和列索引
while ((ch = fgetc(fp)) != EOF) {
    // 確認讀到的是數字字符
    if (ch >= '0' && ch <= '9') {
        G[i][j] = ch - '0'; // 將字符轉換為整數
        j++;
        if (j == n) { // 當列數達到 1023 時，換到下一行
            j = 0;
            i++;
            if (i == k) break; // 已讀取完整矩陣，停止
        }
    }
}
```

```
for(int i = 0; i < 2046; i++){
    for(int j = 0; j < 32; j++){
        if(i < 1023) fscanf(fp, "%d", &s_to_x[i][j]);
        else fscanf(fp, "%d", &x_to_s[i-1023][j]);
    }
    fgets(dump2, 100, fp);
}
```

將ldpc_G_1023.txt存成G矩陣

*因為文檔裡面的矩陣數值之間沒有空格，要注意要用字串的方式存取

將ldpc_H_1023.txt分別存成兩個矩陣

s_to_x[1023][32]

x_to_s[1023][32]

Decoding of LDPC code

```
initiaailize(y);  
for(int count = 0; count < iteration; count++){  
    bottom_up(q,r,s_to_x);  
    top_down(q,r,x_to_s);  
    termination(r,x_to_s);  
    int syndrome = 0;  
    for(int i = 0; i < n; i++){  
        syndrome = 0;  
        for(int j = 0; j < 32; j++){  
            syndrome ^= xhat[s_to_x[i][j] - 1];  
        }  
        if(syndrome != 0) break;  
    }  
    if(syndrome == 0) break;  
}
```

Iteration為最大的迭代次數

在每次count中執行decoding的步驟

並檢驗syndrome，若無任何錯誤，則直接跳出迴圈

Initialize

```
void initailize(double *y){
    //初始化
    for(int i = 0; i < n; i++){
        L[i] = 0;
        for(int j = 0; j < n; j++){
            q[i][j] = 0; //q[m][l]
            r[i][j] = 0; //r[m][l]
        }
    }
    for(int i = 0; i < n; i++){
        L[i] = 2 * y[i] / var;
        //printf("%f ", L[i]);
        for(int j = 0; j < 32; j++){
            q[x_to_s[i][j] - 1][i] = L[i]; //q[m][l] = L[l]
        }
    }
}
```

先建立 $q[1023][1023]$ 以及 $r[1023][1023]$ 並初始化
並計算初始的 $L_{m,l}$ 存進去 $q[m][l]$

Bottom-up

```
void bottom_up(double q[1023][1023], double r[1023][1023], int s_to_x[1023][32]){
    int check, self;
    double resultCHK;
    double qhat[31];

    for(int i = 0; i < n; i++){
        for(int j = 0; j < 32; j++){
            self = s_to_x[i][j]; //找到Sm對應到的所有1
            int num = 0;

            for(int k = 0; k < 32; k++){
                check = s_to_x[i][k];
                if(check != self){
                    qhat[num] = q[i][check - 1]; //把除了自己的q[m][1]存起來
                    num++;
                }
            }

            resultCHK = CHK(&qhat[0], &qhat[1]);
            for(int g = 2; g < 31; g++){
                resultCHK = CHK(&resultCHK, &qhat[g]);
            }
            //if(i==0 && j == 0) printf("%f\n", resultCHK);
            r[i][self - 1] = resultCHK; //r[m][1] = chk
        }
    }
}
```

找到除了自己的 $q[m][l]$ 找出來存入 $qhat[31]$

並計算CHK存入 $r[m][l]$

$$r_{m,l} = CHK_{l' \in L(m) \setminus l}(q_{m,l'})$$

$$CHK(x_1, x_2, x_3 \dots x_n) = CHK((x_1, x_2), x_3) \dots, x_n)$$

CHK

```
double CHK(double *L1, double *L2){
    double temp, delta;
    int sign = ((*L1 > 0) - (*L1 < 0)) * ((*L2 > 0) - (*L2 < 0));
    double min = (fabs(*L1) > fabs(*L2)) ? fabs(*L2) : fabs(*L1);
    temp = (sign > 0) ? min : -min;
    if(decision == 0){ //SPA
        delta = log( (1.0 + exp(-fabs(*L1 + *L2))) / (1.0 + exp(-fabs(*L1 - *L2))) );
        temp = temp + delta;
    }

    return temp;
}
```

Min-Sum Algorithm(MSA)

$$CHK(x_1, x_2) = \text{sgn}(x_1) \text{sgn}(x_2) \min(|x_1|, |x_2|)$$

Sum-Product Algorithm(SPA)

$$CHK(x_1, x_2) = \text{sgn}(x_1) \text{sgn}(x_2) \min(|x_1|, |x_2|) + \Delta(x_1, x_2)$$

$$\text{where } \Delta(x_1, x_2) = \ln\left(\frac{1+e^{-|x_1+x_2|}}{1+e^{-|x_1-x_2|}}\right)$$

Top-down

```
void top_down(double q[1023][1023], double r[1023][1023], int x_to_s[1023][32]){  
  
    int self_var, check_var;  
    for(int i = 0; i < n; i++){  
        for(int j = 0; j < 32; j++){  
            self_var = x_to_s[i][j];  
            double VAR = 0.0;  
            for(int k = 0; k < 32; k++){  
                check_var = x_to_s[i][k];  
                if(check_var != self_var){  
                    VAR += r[check_var - 1][i];  
                }  
            }  
            q[self_var - 1][i] = L[i] + VAR;  
        }  
    }  
}
```

找到除了自己以外的 $r[m][l]$
並計算VAR存入 $q[m][l]$

$$q_{m,l} = L_l + \sum_{m' \in M(L) \setminus m} r_{m',l}$$

Termination

```
void termination(double r[1023][1023], int x_to_s[1023][32]){
    double VAR, total_VAR;
    int node;

    for(int i = 0; i < n; i++){
        VAR = 0.0;
        total_VAR = 0.0;
        for(int j = 0; j < 32; j++){
            node = x_to_s[i][j];
            VAR += r[node - 1][i];
        }
        total_VAR = VAR + L[i];
        xhat[i] = (total_VAR > 0) ? 0 : 1;
        if(i < k) decoded_bits[i] = (total_VAR > 0) ? 0 : 1;
    }
}
```

將所有的 $r[m][l]$ 相加並加上初始的 $L[i]$
若出來的值大於0則 $xhat[i] = 0$
 $xhat$ 為解出來的codeword
並將前781個bit取出來作為decode出來的bit

遇到的困難

- Information bits要一次產生出太多個怕記憶體爆掉，改成每個block產生一次新的information bits，結果改的時候沒改好，debug超久才發現原來是這裡出問題
- Normal的function裡面用的是Sigma不是Variance
- q跟r矩陣抓出來跟存進去的位置一直搞混
- 有碰到其他科考試，沒有做好時間規劃，加上跑simulation需要許多時間，過程中極度焦慮，怕敢不上繳交期限

Simulation (SPA with maximum 100 iterations)

SNR	0.0	0.2	0.4	0.6	0.8	1.0	1.2	1.4	1.6	1.8	2.0	2.2	2.4
Decode Blocks	100	100	100	100	100	100	1000	1000	1000	1000	1000	1000	1000
BER No.	8442	8044	7631	7220	6880	6385	60360	56665	52068	47347	39228	29004	16669
BER rate	1.081E-01	1.030E-01	9.771E-02	9.245E-02	8.809E-02	8.175E-02	7.729E-02	7.255E-02	6.667E-02	6.062E-02	5.023E-02	3.174E-02	2.134E-02
BLER No.	100	100	100	100	100	100	1000	1000	995	971	866	670	391
BLER rate	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.995	0.971	0.866	0.670	0.391

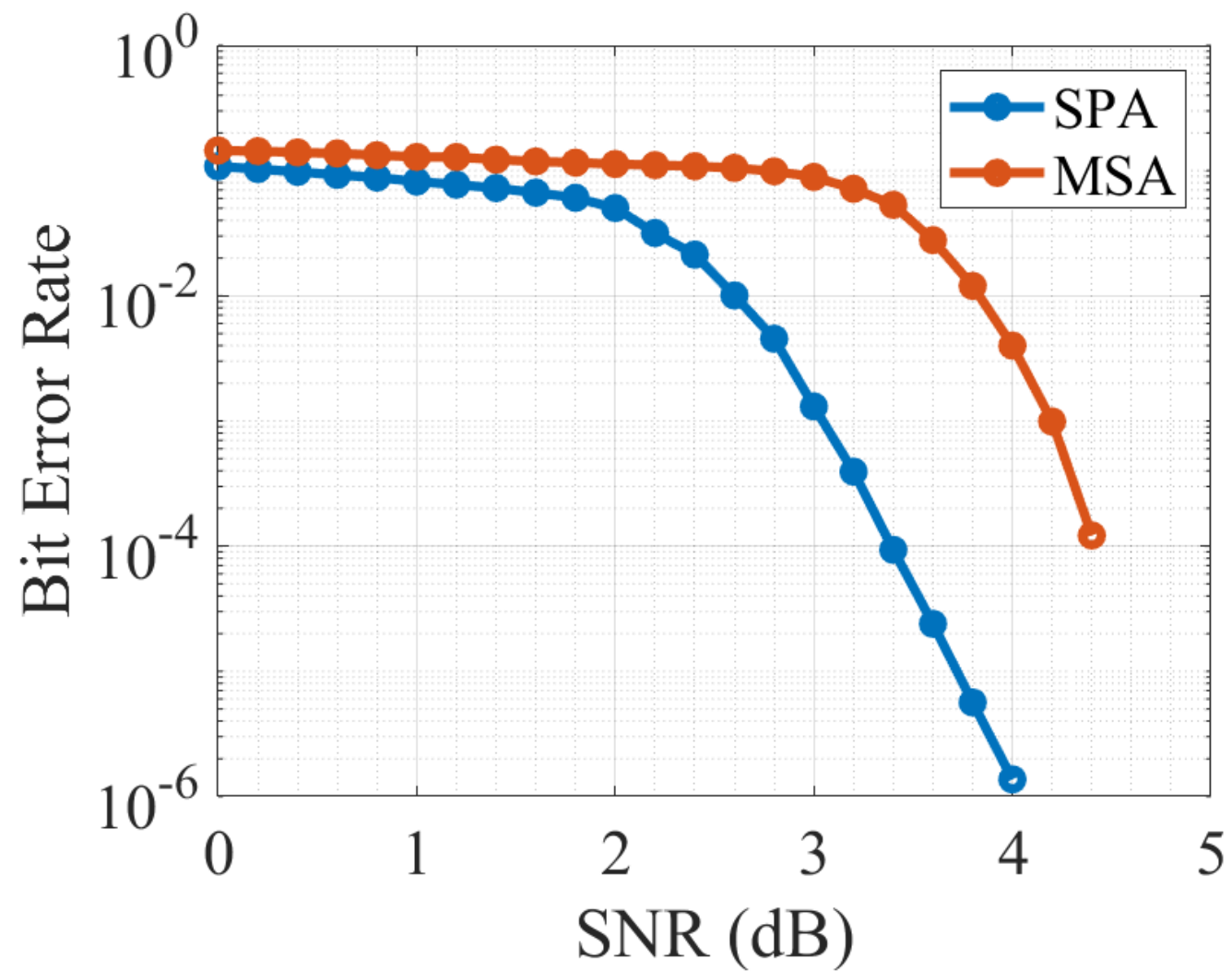
SNR	2.6	2.8	3.0	3.2	3.4	3.6	3.8	4.0
Decode Blocks	1000	1000	10000	10000	10000	10000	10000	100000
BER No.	7864	3547	10201	3070	730	187	44	107
BER rate	1.007E-02	4.542E-03	1.306E-03	3.931E-04	9.347E-05	2.394E-05	5.634E-06	1.370E-06
BLER No.	192	84	238	70	16	4	1	3
BLER rate	0.192	0.084	0.0238	0.0070	0.0016	0.0004	0.0001	0.0003

Simulation (MSA with maximum 100 iterations)

SNR	0.0	0.2	0.4	0.6	0.8	1.0	1.2	1.4	1.6	1.8	2.0	2.2	2.4	2.6	2.8
Decode Blocks	100	100	100	100	100	100	100	100	100	100	100	1000	1000	1000	1000
BER No.	11314	11170	10912	10713	10373	10042	10030	9558	9245	9041	8821	86356	84627	82094	76671
BER rate	1.449E-01	1.430E-01	1.397E-01	1.372E-01	1.328E-01	1.286E-01	1.284E-01	1.224E-01	1.184E-01	1.158E-01	1.129E-01	1.106E-01	1.084E-01	1.051E-01	9.871E-02
BLER No.	100	100	100	100	100	100	100	100	100	100	100	999	998	989	954
BLER rate	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.999	0.998	0.989	0.954

SNR	3.0	3.2	3.4	3.6	3.8	4.0	4.2	4.4
Decode Blocks	1000	1000	1000	1000	10000	10000	10000	10000
BER No.	69573	55926	42031	21733	93667	31221	7736	951
BER rate	8.908E-02	7.161E-02	5.382E-02	2.783E-02	1.199E-02	3.998E-03	9.905E-04	1.218E-04
BLER No.	879	718	541	285	1240	419	100	13
BLER rate	0.879	0.718	0.541	0.285	0.124	0.0419	0.010	0.0013

Simulation



Simulation

